bk-automa 仕様書

ブラウザ自動化拡張機能の
技術仕様と機能概要

github.com/BKStock/bk-automa

Forked from AutomaApp/automa

Version 1.30.00 Latest

社内向け技術資料

bk-automaはブロック接続型のブラウザ自動化ツールである

bk-automaは、AutomaApp/automaをフォークしたブラウザ拡張機能です。

ビジュアルなブロック接続インターフェースを通じて、プログラミング不要で複雑なブラウザ操作を自動化できます。

- Chrome Web Store
- Firefox Add-ons
- マーケットプレイスで共有可能

- ノーコード/ローコードでワークフロー構築
- フォーム自動入力・繰り返しタスク
- スクリーンショット取得・Webスクレイピング
- ワークフローのスケジュール実行
- Chrome Extension Builder (CEB) 対応

技術スタックはVue 3 + Webpack + WebExtension APIで構成される

フロントエンド

- **Vue 3.4.38**
  Composition API / Options API
- **Pinia / Vuex**
  状態管理
- **TailwindCSS 3**
  ユーティリティファーストCSS
- **Vue Flow**
  ノードベースUI / ワークフローエディタ
- **CodeMirror 6 / Tiptap**
  コードエディタ / リッチテキスト

ビルド・開発環境

- **Webpack 5.76**
  モジュールバンドラー
- **pnpm**
  高速なパッケージマネージャー
- **Node.js 20.11.1**
  Voltaによるバージョン管理
- **Babel / ESLint**
  トランスパイラ / 静的解析

ブラウザ拡張

- **Manifest V3 / V2**
  Chrome (V3) / Firefox (V2互換)
- **webextension-polyfill**
  クロスブラウザAPI互換性確保
- **Dexie.js**
  IndexedDBラッパー / ローカルデータ

アーキテクチャは5層構造で設計されている

UI層
ダッシュボード、ポップアップ、ワークフローエディタ

`src/newtab/, src/popup/, src/components/`

ストア層
ワークフロー・ユーザー・パッケージの状態管理 (Pinia)

`src/stores/`

ワークフローエンジン層
ブロック実行、ワーカー管理、テンプレート処理

`src/workflowEngine/`

バックグラウンド層
イベントリスナー、API通信、トリガー管理

`src/background/`

コンテンツスクリプト層
DOM操作、要素選択、ページ内ブロック実行

`src/content/`

データフロー

1. **設計 (UI層)**
   ユーザーがUIでワークフローを設計し、JSONとして保存
2. **初期化 (Engine層)**
   WorkflowEngineが起動し、実行計画を策定
3. **実行 (Engine/Worker)**
   WorkflowWorkerがブロックを順次実行
4. **操作 (Content層)**
   コンテンツスクリプトが実際のDOM操作を実行
5. **保存 (Store/BG層)**
   実行結果をストレージに保存し、ログを記録

ワークフローエンジンがブロック実行の中核を担う

WorkflowEngine

Manager / Orchestrator

- ワークフローの初期化・実行・停止・破棄を一元管理
- ノードとエッジからDAG（有向非巡回グラフ）を構築
- WorkflowWorkerを生成し、並列実行を制御
- 参照データ（変数、テーブル、ループデータ、シークレット）の管理
- 実行ログの記録とAPIへのレポート送信

WorkflowWorker

Executor / Worker

- 個々のブロックを順次実行
- ブロックハンドラーの呼び出しとタイムアウト管理
- テンプレートエンジンによるデータ置換（Mustache記法）
- アクティブタブの管理とフレーム切り替え
- ブレークポイントによるデバッグ支援とエラー時のリトライ処理

60種類以上のブロックが7カテゴリに分類されている

Browser 13

- Active Tab
- New Tab
- Switch Tab
- New Window
- Go Back
- Go Forward
- Close Tab
- Take Screenshot
- Browser Event
- Handle Dialog
- Handle Download
- Reload Tab
- Get Tab URL

Interaction 12

- Click Element
- Get Text
- Scroll Element
- Attribute Value
- JavaScript Code
- Trigger Event
- Switch Frame
- Upload File
- Hover Element
- Save Assets
- Press Key
- Create Element

Conditions 6

- Repeat Task
- Element Exists
- While Loop
- Loop Data
- Loop Elements
- Loop Breakpoint

Data 8

- Insert Data
- Delete Data
- Get Log Data
- Slice Variable
- Increase Variable
- RegEx Variable
- Data Mapping
- Sort Data

General 8

- Trigger
- AI Workflow
- Execute Workflow
- Export Data
- Blocks Group
- Wait Connections
- Workflow State
- Parameter Prompt

Online Services 3

- Google Sheets
- Google Sheets (GDrive)
- Google Drive

Others 7

- Webhook
- Clipboard
- Notification
- Cookie
- Proxy
- Delay
- Block Package

10種類のトリガーでワークフロー実行を開始できる

Manual
手動で実行

Interval
一定間隔で繰り返し

Cron Job
Cron式によるスケジュール

Specific Date
特定の日時に実行

Specific Day
特定の曜日に実行

Visit Web
Webサイト訪問時

Shortcut
キーボードショートカット

Context Menu
右クリックメニュー

On Startup
ブラウザ起動時

Element Change
DOM要素の変更検知

スケジュール機能

Technical Implementation

高度なスケジュール管理を実現するために、`cron-parser` ライブラリを採用し、複雑なCron式に対応しています。バックグラウンドでのタイマー管理には `browser.alarms` APIを利用し、ブラウザのリソース消費を抑えつつ正確な実行タイミングを保証します。

データ管理はIndexedDBとブラウザストレージの二層構造

IndexedDB (Dexie.js)

tablesData	収集したテーブルデータ
tablesItems	テーブルのメタ情報
variables	ユーザー定義変数 (永続化)
credentials	認証情報・シークレット

Browser Storage

workflows	ワークフロー定義
settings	ユーザー設定
state:*	実行状態 (reuseLastState)
workflowQueue	実行待ちキュー

実行時参照データシステム (Reference Data)

- variables
- table
- secrets
- loopData
- globalData
- googleSheets

ブラウザ権限モデルはManifest V3に準拠している

必須権限 (permissions)

tabs
タブの操作・情報取得

storage
ローカルデータの永続化

activeTab
アクティブタブへのアクセス

scripting
コンテンツスクリプトの動的注入

alarms
スケジュール実行のタイマー

webNavigation
ページ遷移の検知

debugger
Chrome DevTools Protocol連携

proxy
プロキシ設定の制御

offscreen
オフスクリーンドキュメント処理

unlimitedStorage
ストレージ容量制限の解除

オプション権限

cookies / downloads
クッキー操作 / ダウンロード管理

contextMenus
右クリックメニューの追加

clipboardRead / notifications
クリップボード読み取り / 通知

コンテンツスクリプト

`contentScript.bundle.js`
全URLに対して注入 (document_start, 全フレーム)

`webService.bundle.js`
automa.site ドメインに追加注入

ダッシュボードUIは7つの主要画面で構成される

ワークフロー一覧	`/workflows`	管理・作成・検索
ワークフロー編集	`/workflows/:id`	Vue Flow ビジュアルエディタ
スケジュール	`/schedule`	スケジュール実行の管理
パッケージ	`/packages`	再利用可能なブロック管理
ストレージ	`/storage`	テーブルデータの閲覧・管理
ログ	`/logs/:id`	実行ログの詳細表示
設定	`/settings`	一般・バックアップ・ショートカット

Additional Features

Recording機能
操作を記録してワークフロー変換

Element Selector
クリックで要素セレクタを取得

共有ワークフロー
チーム間でのワークフロー共有

コマンドパレット
ショートカットによる素早い操作

外部連携とAI機能で自動化の幅を拡張している

Google連携

- Google Sheets API：スプレッドシートの読み書き
- Google Drive API：ファイルのアップロード・管理
- Drive経由でのスプレッドシート操作連携

Webhook / HTTP Request

- 任意のHTTPリクエスト送信（GET/POST/PUT/DELETE）
- カスタムヘッダー・ボディの設定とJSON対応
- レスポンスデータの変数への格納と再利用

AI Workflow機能

- AI-Powerプラットフォームとのシームレスな連携
- ワークフロー内でのAIモデル呼び出しと推論実行
- 入力データのテンプレート置換と結果の構造化保存

その他の連携機能

- Clipboard操作（テキストの読み取り/書き込み）
- ブラウザ通知機能によるユーザーへのフィードバック
- Cookie管理（取得/設定/削除）とプロキシ切り替え

開発・ビルド・デプロイの手順

開発環境セットアップ

1. リポジトリをクローン
   `git clone ...`
2. 認証キーファイルを作成
   `src/utils/getPassKey.js`
3. 依存関係をインストール
   `pnpm install`
4. 開発サーバー起動
   `pnpm dev`

ビルドコマンド

`$ pnpm build`	Chrome向けビルド
`$ pnpm build:firefox`	Firefox向けビルド
`$ pnpm build:zip`	ZIP生成
`$ pnpm lint`	コード品質チェック

ローカルインストール

Chrome

1. **chrome://extensions**
2. 開発者モード ON
3. 「パッケージ化されていない拡張機能を読み込む」

Firefox

1. **about:debugging**
2. 「一時的なアドオンを読み込む」
3. manifest.jsonを選択

ライセンスと今後の展望

ライセンス形態

GNU AGPL v3.0

または

Automa Commercial License

(商用利用時は要確認)

2,121	100	22
Commits	Releases	Branches

Vue 57.6% **JS 41.8%**

今後の活用ポイント

定型業務の自動化

社内オペレーションの効率化

Web スクレイピング

競合調査・データ収集

レポート自動生成

Google Sheets連携活用

AI Workflow

高度な判断を伴う自動化

チーム共有

ワークフローの資産化